

Sesión 1

Del modelo al servicio: despliegue y APIs REST productivas



Jaime A. Riascos-Salas

CEIA · MIA — Facultad de Ingeniería, Universidad de Buenos Aires

Antes de arrancar



Descripción y objetivos

Qué es MLOps II y para qué



Programa y fechas

Las 8 sesiones del bimestre



Metodología y evaluación

Mini-TPs + trabajo práctico integrador



El docente

Quién dicta el curso



Sesión 1

Del modelo al servicio: APIs REST

¿De qué se trata MLOps II?

Proporcionar los conceptos y herramientas para desarrollar productos de software de Machine Learning de manera escalable y con buenas prácticas, y para operar el ciclo de vida de los modelos (MLOps) disponibilizándolos en entornos productivos.

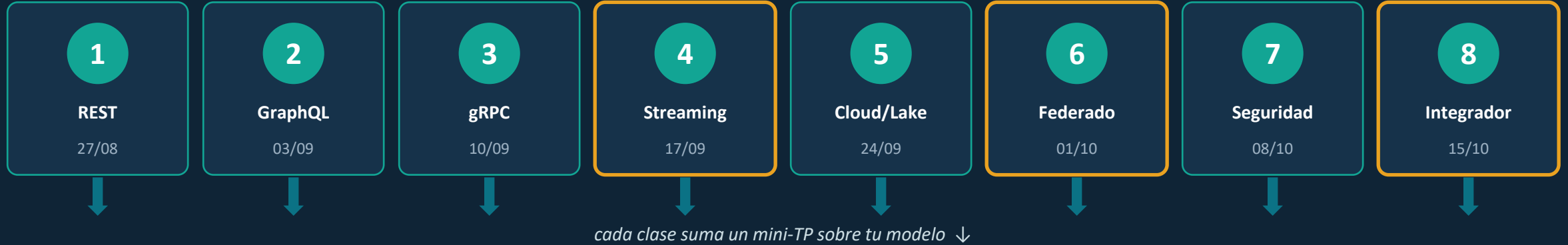
Continúa MLOps I. Se centra en cómo se comunica y opera un modelo en producción: flujo de datos, REST/GraphQL/gRPC, streaming, nube y Data Lakes, aprendizaje federado, y seguridad/operación/gobernanza.

Cronograma del bimestre

Sesión	Fecha	Tema
1	27/08/26	Del modelo al servicio: APIs REST productivas (online asíncrona)
2	03/09/26	GraphQL en MLOps + grafos de linaje
3	10/09/26	gRPC para microservicios de ML
4	17/09/26	Streaming e inferencia en tiempo real · Hito TP #1
5	24/09/26	Nube y Data Lakes para MLOps
6	01/10/26	Aprendizaje Federado · Hito TP #2
7	08/10/26	Seguridad, operación y gobernanza (online asíncrona)
8	15/10/26	Taller integrador y defensa del TP

Cada jueves. El material se publica semana a semana.

Cada sesión construye el proyecto integrador



PROYECTO INTEGRADOR

Plataforma de predicción en tiempo real — "ML Models & Something More Inc."

Grupal (2–6). Niveles: local (6–8) · contenedores (8–10).

Hito #1 S4 · arquitectura

Hito #2 S6 · checkpoint

Defensa S8 · presentación

Cómo se aprueba



Durante el curso

- Mini-TP corto y evaluable por clase (S1–S7)
- Siempre sobre tu propio modelo
- Se valora la participación activa
- Cada mini-TP es un ladrillo del integrador



TP integrador (grupal)

Equipos de 2–6 personas.

- Nivel local: nota 6–8.
- Nivel en contenedores: nota 8–10.
- Hitos: arquitectura (S4), checkpoint (S6), defensa (S8).

Herramientas: Python · uv · MLflow · Jupyter · GitHub · Docker · Apache Airflow. Repo: github.com/JARS29/MLOPs2_2025_UBA

Jaime A. Riascos-Salas



Formación

- Ingeniero Mecatrónico (2016 — ITM, Colombia).
- MSc en Ciencias de la Computación (2019 — UFRGS, Brasil).
- Profesor Asistente / Data Scientist (2019–2023).
- Research Assistant (2024) — Potsdam University.
- PhD student — Potsdam Embodied Cognition Group (PECoG).

Investigación

Interacción humano-robot multimodal (humanoides), integrando EEG (BCI), GSR y ritmo cardíaco para decodificar estados mentales con Machine y Deep Learning, cognición corporizada y robótica.

jandresrsalas@gmail.com

Agenda de hoy

10'



Apertura y repaso

Ciclo de vida del modelo · de MLOps I a II

25'



Teoría 1 — Despliegue de modelos

Patrones, estrategias, online/batch, dónde corre

30'



Teoría 2 — APIs, microservicios y REST

API, protocolos, HTTP/CRUD, códigos, OpenAPI

20'



Diseño productivo e implementación

Contratos, versionado, FastAPI y cómo consumir

15'



Demo práctica guiada

Niveles 0→3 de una ML API + el problema del CORS

10'



Mini-TP 1 y cierre

Serví tu propio modelo · puente a la Sesión 2

Construimos, clase a clase, la plataforma de ML de una empresa

"ML Models & Something More Inc." ofrece predicción en tiempo real. Cada sesión agrega una capa a la misma arquitectura de referencia. Hoy: el primer canal de comunicación.

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

Hoy incorporamos:



Servicio de inferencia

FastAPI expone el modelo por REST



Validación de contratos

Entradas y salidas verificadas (Pydantic)

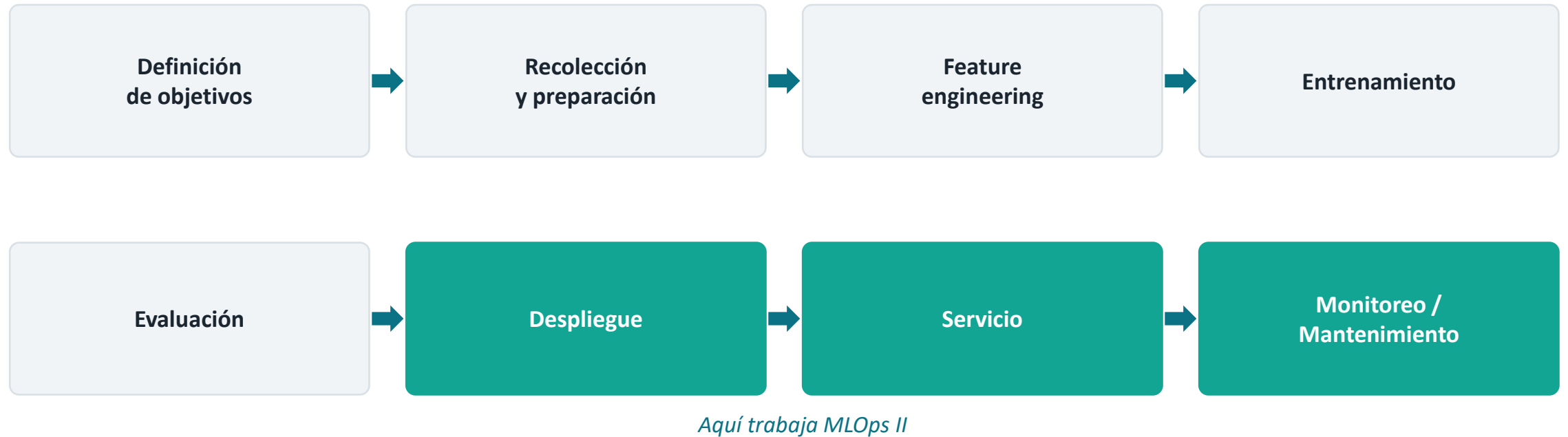


Empaquetado

El servicio corre en un contenedor Docker

Ciclo de vida de un proyecto de ML

El modelo entrenado es solo la mitad del camino. MLOps II vive en la etapa sombreada: desplegar, servir, monitorear y mantener.



De MLOps I a MLOps II

Veníamos de (MLOps I)

- Entrenamiento reproducible y tracking (MLflow)
- Orquestación de pipelines (Airflow)
- Versionado de datos y modelos
- Registro y empaquetado del artefacto

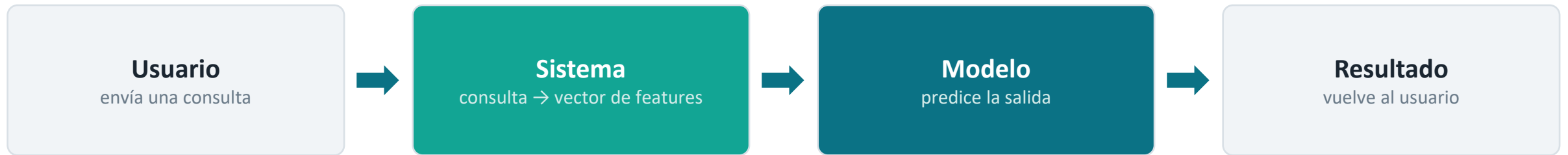
Vamos hacia (MLOps II)

- Cómo se comunica el modelo: REST, GraphQL, gRPC
- Datos en movimiento: streaming e inferencia RT
- Dónde viven datos y artefactos: nube y Data Lakes
- Entrenar sin centralizar: aprendizaje federado
- Proteger, operar y gobernar en producción

El foco pasa del entrenamiento a la disponibilización en producción.

¿Qué significa desplegar un modelo?

Desplegar es dejar el modelo disponible para que acepte consultas en producción. La consulta se transforma en un vector de características, el modelo predice y el resultado vuelve al usuario.



Todo lo que sigue en la materia es sobre este canal: cómo entra la consulta, cómo se comunica y cómo se opera.

Patrones de despliegue



Estático

Como parte de un paquete de software instalable. El modelo viaja con la app.



Dinámico en el dispositivo

Se actualiza en el dispositivo del usuario (móvil/edge), sin reinstalar la app.



Dinámico en un servidor

El modelo vive en un servidor y se sirve por la red. El foco de esta materia.



Transmisión de modelo

El modelo se envía/actualiza por streaming a quien lo necesita.

Estrategias de despliegue

No solo importa dónde corre el modelo, sino cómo lo ponemos en producción sin romper nada.

- | | | |
|---|---------------------------------------|---|
| 1 | Despliegue único | Se reemplaza el modelo viejo por el nuevo de una vez. Simple, pero arriesgado. |
| 2 | Despliegue silencioso (shadow) | El nuevo modelo corre en paralelo sin afectar al usuario; se comparan resultados. |
| 3 | A/B testing | Se divide el tráfico entre dos modelos para medir cuál funciona mejor. |
| 4 | Canary release | Se libera el nuevo modelo a un pequeño % de usuarios y se escala si va bien. |
| 5 | Bandidos (multi-armed) | Se asigna tráfico dinámicamente al modelo que mejor rinde en el momento. |

Predicción online vs batch



ONLINE (on-line)

- Sincrónica: el cliente pide y espera
- Una (o pocas) muestras por request
- Prioridad: baja latencia y disponibilidad
- La forma básica es una REST API
- Ej.: scoring de una transacción al vuelo



BATCH (por lotes)

- Asincrónica: se procesa un lote grande
- Cientos/miles de muestras a la vez
- Se guardan en tablas SQL / data warehouse
- Prioridad: throughput y costo; escala horizontal
- Ej.: recomputar scores de todos de noche

Muchos sistemas usan ambos: online para responder, batch para precomputar.

¿Dónde corre el servicio?



Máquina virtual

Servicio web típico, conceptualmente simple.

- + Simple de entender
- Mantener servidores, overhead, latencia



Contenedores

Aislado y liviano; un orquestador escala el clúster.

- + Eficiente, escala a cero
- Requiere más experiencia



Serverless

Subir un zip/imagen; el proveedor ejecuta y escala.

- + Sin aprovisionar, se paga por uso
- Límites de tamaño/RAM, sin GPU

El memo de Bezos (Amazon, 2002)

1. Todos los equipos expondrán sus datos y funcionalidades a través de interfaces de servicio.
2. Los equipos se comunican entre sí únicamente a través de esas interfaces.
3. Nada de enlaces directos, lecturas de la BD de otro equipo ni memoria compartida: solo llamadas por la red.
4. No importa la tecnología (HTTP, Corba, Pubsub, ...).
5. Toda interfaz debe diseñarse para poder externalizarse a desarrolladores externos.

Este mandato impulsó la estandarización de APIs y microservicios que hoy usamos.

¿Qué es una API?

Una API (interfaz de programación de aplicaciones) es la puerta de enlace que permite a un programa comunicarse con otro. Habilita dos cosas:



Acceso a los datos

Consultar la información que expone una aplicación.



Uso de la funcionalidad

Invocar lo que la aplicación sabe hacer (p. ej. predecir).

Gracias a las APIs, dispositivos, aplicaciones y sitios web se comunican entre sí para resolver tareas de negocio o de operación. Para nosotros: es como el mundo consume nuestro modelo.

Los protocolos de API a lo largo del tiempo

Protocolo	Año	Formato	Característica clave
SOAP	1990s	XML	Muy usado y establecido
REST	2000	XML, JSON y otros	Formato de datos flexible ← hoy
JSON-RPC	2005	JSON	Fácil de implementar
Thrift	2007	JSON o binario	Adaptable a muchos casos
gRPC	2015	Protobuf / binario	Definir cualquier tipo de función ← Sesión 3
GraphQL	2015	JSON	Consulta flexible ← Sesión 2

Empezamos por REST (hoy) y recorreremos GraphQL y gRPC en las próximas sesiones.

Monolito vs. microservicios (SOA)

Monolítico

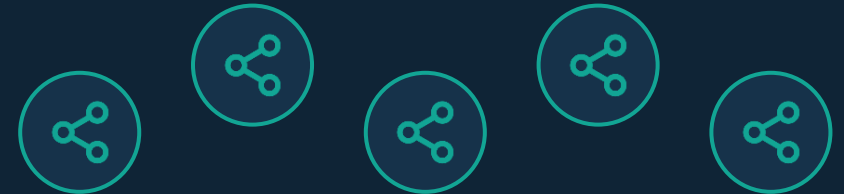
Toda la funcionalidad agrupada en una gran aplicación. Simple al inicio; difícil de escalar y de que varios equipos trabajen en paralelo.



Un bloque, una base de datos

Microservicios (SOA)

La app se divide en servicios pequeños e independientes que se comunican por APIs. Principios: interoperabilidad, acoplamiento flexible, abstracción y granularidad.



Servicios chicos + una API que los orquesta

Un proyecto de ML, con sus muchos pasos, se beneficia de esta arquitectura: la API es su consecuencia natural.

REST: el estándar de facto para servir ML

REST (Representational State Transfer) es el estándar de oro para servicios web y también para servir modelos de ML. Beneficios:



Predicciones al vuelo

Responde a múltiples usuarios en tiempo real.



Escala horizontal

Sumar instancias detrás de un balanceador de carga.



Varios modelos

Combinar modelos usando distintos endpoints.



Separa entornos

Aislar el entorno del modelo del entorno del usuario.



Habilita microservicios

Equipos trabajando de forma independiente.

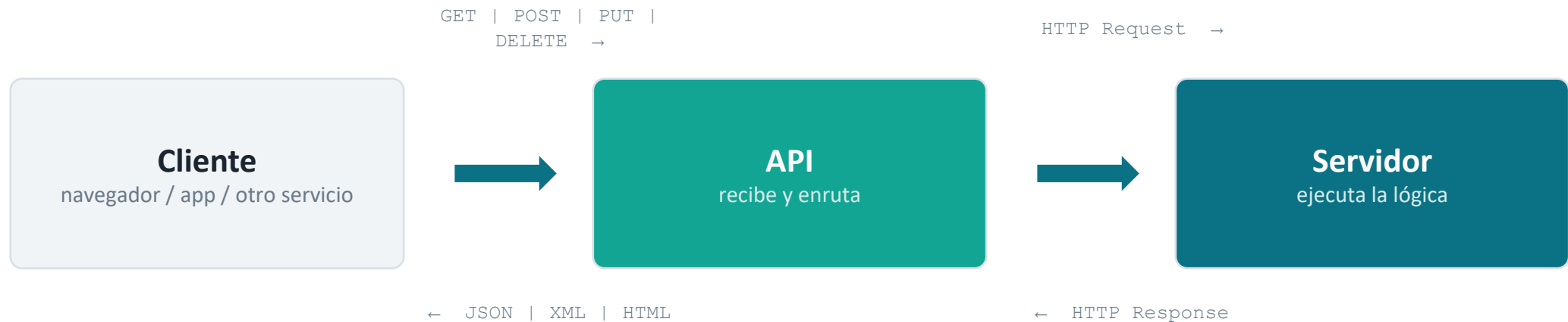


Universal

HTTP + JSON: lo habla cualquier lenguaje.

HTTP: el modelo cliente–servidor

HTTP es el protocolo de comunicación de la web. Un cliente hace una solicitud; el servidor responde. Una API REST se apoya en HTTP.



Métodos HTTP ↔ CRUD en ML

Método	CRUD	Propósito en una aplicación de ML
GET	Read	Leer/recuperar un recurso (p. ej. info o metadatos del modelo)
POST	Create	Crear un recurso — en ML: obtener una inferencia del modelo
PUT	Update/Replace	Reemplazar un recurso — en ML: enviar un archivo (p. ej. imagen)
PATCH	Update/Modify	Actualizar parcialmente un recurso
DELETE	Delete	Eliminar un recurso — p. ej. un microservicio o artefacto

En la práctica, servir una predicción es un POST: enviamos las features y recibimos la salida.

Códigos de estado HTTP

2xx

Operación exitosa

3xx

Redirección

4xx

Error del cliente

5xx

Error del servidor

Un contrato bien hecho devuelve 422 (entrada inválida) antes de tocar el modelo, y 200 con la predicción cuando todo va bien.

Endpoints: las URLs públicas del servicio

Una API REST expone un conjunto de URLs (endpoints) que las aplicaciones cliente usan para acceder a los recursos.

Método	Endpoint	Descripción
GET	/customers	Lista de clientes
GET	/customers/{id}	Un cliente en particular
POST	/customers	Crea un nuevo cliente
POST	/v1/predict	Inferencia del modelo (nuestro caso)
DELETE	/customers/{id}	Borra un cliente

Se accede como: `https://api.example.com/v1/predict`

OpenAPI: el contrato legible de la API

OpenAPI es un formato estandarizado para describir una API REST, entendible por humanos y máquinas. Habilita validación, pruebas y documentación interactiva.



Endpoints y operaciones

Qué rutas hay y qué hace cada verbo



Entrada y salida

Parámetros y esquema de cada operación



Autenticación

Cómo se protege el acceso



Documentación

Docs para el desarrollador (Swagger UI)

FastAPI genera OpenAPI + Swagger UI automáticamente en /docs.

Diseñar una REST API para ML



Endpoints con intención

/predict, /predict/batch, /health, /metadata.



Validación estricta

Rechazá entradas mal formadas antes del modelo (422).



Versionado

/v1/... y versión del modelo en la respuesta.



Sin estado

Cada request se basta solo; escalás con réplicas.



Observabilidad

Log de latencia, estado y versión por request.



Portabilidad

Servicio + dependencias en una imagen reproducible.

Validación de entrada con contratos

El modelo espera exactamente sus features. Un contrato tipado corta de raíz el 'garbage in, garbage out'.

```
from pydantic import BaseModel, Field

class Caracteristicas(BaseModel):
    feature_1: float = Field(gt=0)
    feature_2: float = Field(ge=0)

class Resultado(BaseModel):
    clase_predicha: int
    etiqueta: str
# FastAPI devuelve 422 si no cumple
```

Qué te da gratis

- Tipos y rangos garantizados antes de predecir
- Mensajes de error precisos para quien consume
- Documentación OpenAPI automática
- Un único lugar donde vive el contrato
- `response_model` valida también la salida

Versionar la API y el modelo



Versión de API

- Ruta con versión: /v1/predict, /v2/predict
- Cambios que rompen → nueva versión
- Los clientes migran cuando pueden



Versión de modelo

- La respuesta declara qué modelo predijo
- Se resuelve desde el registry (MLflow)
- Permite trazar y reproducir cada predicción

```
# nivel 3 de la práctica: routers versionados
router_v1 = APIRouter(prefix="/v1")    router_v2 = APIRouter(prefix="/v2")
@router_v2.post("/predecir") -> { "version": "2.0", "prediccion": ... }
```

Estado, escalabilidad y qué monitorear



Sin estado → réplicas

Cada request independiente; escalás sumando instancias.



Modelo en memoria

Cargalo una vez al iniciar, nunca por request.



Qué monitorear

Latencia p50/p95, throughput, tasa de error y drift de entradas.

La operación y la gobernanza las profundizamos en la Sesión 7. Hoy dejamos los ganchos puestos.

Implementar APIs en Python

Hay varias herramientas; usamos FastAPI.

Flask

Micro-framework simple

Django

Full-stack, más pesado

FastAPI

Moderno, para APIs de datos/ML

¿Por qué FastAPI?

- Basado en estándares abiertos (OpenAPI).
- Documentación interactiva automática con Swagger UI (/docs).
- Validación de datos con Pydantic (Python 3.8+).
- Seguridad y autenticación integradas.
- Todo funciona por defecto y es fácilmente actualizable.

No solo crear APIs: también consumirlas

```
import requests

payload = {'feature_1': 12.5, 'feature_2': 4.2}
r = requests.post(
    'http://localhost:8004/predecir_estructurado',
    json=payload)
print(r.status_code, r.json())
```

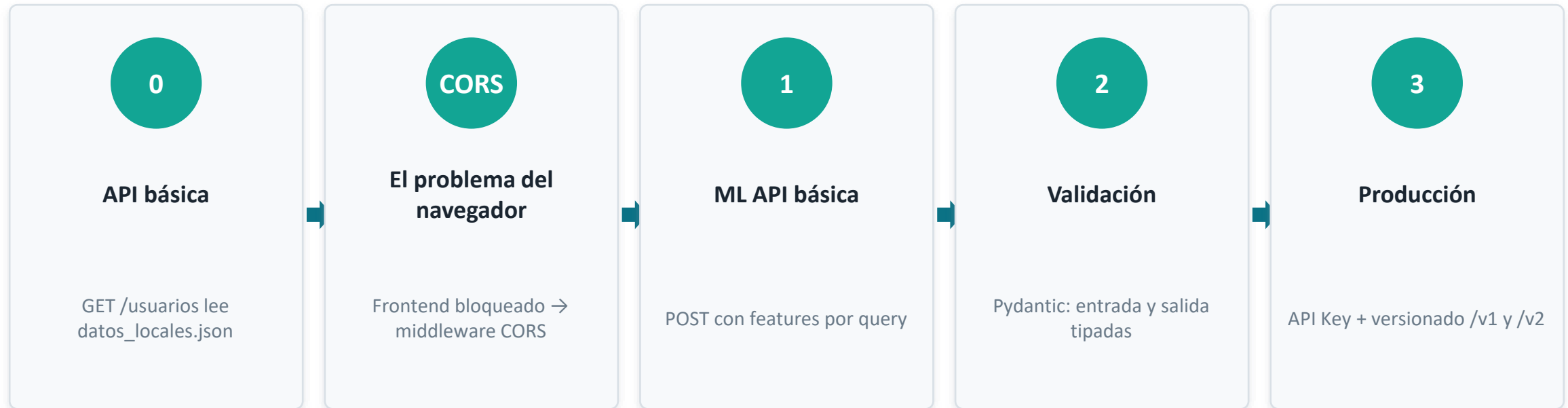
Para el usuario final

La librería requests abstrae el HTTP. Y para una interfaz visual rápida de prototipo, se pueden usar gradio o streamlit, que dejan que el usuario interactúe con el modelo sin escribir código.

En la demo probamos la API desde Python (requests) y desde un frontend HTML.

Qué construimos (online)

Recorremos el notebook API_MLOPS2.ipynb, que va escribiendo y levantando cada servicio. El paso a paso completo, con puertos y comandos, está en la Guía de la práctica.



Todos los notebooks corren de punta a punta y están en clase1/Practica del repo.

De juguete a producción, en 3 niveles

Nivel 1

POST con features por query. Sin validación. Muestra la idea.

`api_ml_nivel1.py` · 8003

Nivel 2

Pydantic: esquema de entrada y de salida. Errores 422 claros.

`api_ml_nivel2.py` · 8004

Nivel 3

API Key (X-API-KEY) + versionado /v1 y /v2. Listo para producción.

`api_ml_nivel3.py` · 8080

Además, la demo del CORS: por qué un frontend en el navegador se bloquea y cómo el middleware CORS lo resuelve.

Empaquetar en Docker (nivel del TP)

```
# Practica/Dockerfile (usa uv)
FROM python:3.11-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/
WORKDIR /app
COPY requirements.txt .
RUN uv pip install --system -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["uvicorn", "api_ml_nivel3:app",
     "--host", "0.0.0.0", "--port", "8080"]
```

Cómo se ejecuta

- Vive en Practica/ (Dockerfile + requirements.txt)
- uv instala las deps (rápido) en la imagen
- `docker build -t mlapi-nivel3 .`
- `docker run -p 8080:8080 mlapi-nivel3`
- Base del nivel 'contenedores' del TP

Errores comunes (y cómo evitarlos)

Fallas de laboratorios de ediciones anteriores, para que no se repitan.



Cargar el modelo dentro del endpoint

Cargalo una vez al iniciar la app.



Features en distinto orden que en el entren.

Fijar el orden en el contrato; testealo.



Puertos o rutas hardcodeadas

Variables de entorno y rutas relativas.



Versiones sin fijar → 'a mí me anda'

uv.lock / requirements con versiones exactas.



Sin manejo de errores

422 en entrada inválida, 500 controlado.



MINI-TP 1 · INDIVIDUAL · ENTREGA ESTA SEMANA

Crea tu propio modelo con FastAPI

Consigna

- Toma tu modelo de Aprendizaje de Máquina (o uno simple).
- Crear un contrato Pydantic con las features reales.
- Exponer `/v1/predict (online)` y `/health`.
- Devolver la predicción y la versión del modelo.
- Escribir un cliente con un caso válido y uno inválido.

Se evalúa

- Corre de punta a punta
- La validación rechaza entradas malas
- Contrato = features del modelo
- Código en el repo, con README

Cimiento del TP integrador: sobre este servicio irán GraphQL, gRPC, streaming, Data Lake y seguridad.

Dónde quedó la plataforma

REST

GraphQL

gRPC

Streaming

Cloud/Lake

Federado

Seguridad

Integra

Hoy logramos

- Entendimos el despliegue de punta a punta
- El modelo vive como servicio REST
- Con contratos validados y versionado



Próxima · Sesión 2

GraphQL y grafos de linaje: cuando el cliente necesita consultar metadatos de experimentos y modelos con flexibilidad, REST se queda corto.

Para la próxima: tené tu Mini-TP 1 corriendo. Lo vamos a extender.

Referencias y recursos

Bibliografía

- Designing Machine Learning Systems — Chip Huyen (O'Reilly): model deployment y prediction service.
- Machine Learning Engineering with Python — Andrew P. McMahon (Packt).
- Practical MLOps — Noah Gift, Alfredo Deza (O'Reilly).

Herramientas y código

- FastAPI — fastapi.tiangolo.com · Pydantic — docs.pydantic.dev · requests, gradio, streamlit.
- Repositorio del curso: github.com/JARS29/MLOPs2_2025_UBA (clase1/Practica).
- Proyecto de referencia: github.com/facundolucianna/amq2-service-ml (branch example_implementation).